From Simulated Slice Admission Control to a Live O-RAN Testbed: Realization and Congested, URLLC-Prioritized Evidence Toward a QoE-Aware Framework

Manoj Prasad Kunasegran, Wai Leong Pang, and Swee King Phang

TODO(MANOJ): author order/inc carries the same authorship before

School of Engineering, Faculty of Innovation & Technology, Taylor's University, Subang Jaya, 47500, Selangor, Malaysia TODO(MANO

***Abstract*—Our own prior work [1], [2] formulated deep reinforcement learning (DRL)-based slice admission control (SAC) and joint SAC–load-balancing (SAC–LB) for O-RAN networks and validated both in simulation, reporting URLLC SLA satisfaction of ≈99.5% (single-gNB SAC) and 99.63/88.41/97.64% per-slice (multi-gNB SAC–LB). Our subsequent systematic review of 2024–2026 ML-driven network-slicing optimisation [3] identified five recurring gaps across 67 core studies – QoE inference, topology awareness, privacy preservation, *validated deployment*, and scalability – and proposed a QoE-aware GAT–MARL–FL framework to close them, explicitly reserving live O-RAN execution as future work. This paper targets the validated-deployment gap directly: we realize the SAC admission-control MDP from [1], [2] on a real OpenAirInterface (OAI) gNB with its native RIC-free E2 control loop – not ns-3/ns-O-RAN simulation – across three slices (eMBB, URLLC, mMTC), and report the control-surface mismatch this exposes: the real E2 agent has no per-flow accept/reject primitive, only a per-slice PRB ratio ceiling, so admission must be realized as *ceiling nudging*, and the ratio-to-PRB mapping this ceiling controls is empirically non-linear. Across 45 live episodes/arm, DQN and A2C under both SLA-only and QoE-aware rewards hold $100.0 \pm 0.0\%$ SLA compliance against a static baseline's seed-dependent $73.6 \pm 18.6\%$ (worst episode 0.0%). We additionally report a preliminary, offline-only follow-up – built after diagnosing that the offline training environment never modelled slices as competing for a shared PRB pool – demonstrating that a DQN policy trained with contention-aware, QoE-shaped reward preserves URLLC SLA compliance ($22.6\% \rightarrow 39.0\%$) by actively trading off eMBB ($\rightarrow 7.6\%$) and mMTC ($\rightarrow 7.7\%$) under genuine cross-slice PRB scarcity – the same URLLC-first prioritization goal as [1], [2], now shown under real contention dynamics rather than isolated per-slice utility. We report what still does not hold up (sim-to-real transfer is informative for eMBB only, not URLLC/mMTC; within-episode demand variation collapses all arms) as plainly as what does, and position this paper's evidence against the five validation directions our review's proposed framework calls out before any deployment claim.**

***Index Terms*—O-RAN, network slicing, deep reinforcement learning, admission control, quality of experience, URLLC prioritization, live testbed**

I. Introduction

Network slicing lets a single physical 5G/O-RAN deployment serve heterogeneous service classes (eMBB, URLLC, mMTC) under per-slice SLA guarantees, but conventional threshold-based control lacks the context-awareness to allocate PRBs adaptively as demand shifts [3]. DRL is a natural fit, and our own prior work took this approach: [1] formulated single-gNB SLA-aware admission control (SAC) balancing URLLC/eMBB congestion, and [2] extended this to a third slice (mMTC) and a multi-gNB load-balancing term. Both were validated entirely in simulation. Our systematic review of the 2024–2026 literature [3] (6,286 records screened to 67 core studies) found this pattern holds across the field: the majority of DRL/MARL/GNN-based slicing optimisation is validated in simulation, under fixed topology, with deferred live deployment – crystallized as the review's *validated-deployment gap*, one of five recurring gaps (alongside QoE inference, topology awareness, privacy preservation, and scalability) that motivate the review's proposed QoE-aware GAT–MARL–FL framework.

This paper closes the validated-deployment gap for the SAC formulation specifically: we realize [1], [2]'s admission-control MDP on a real OAI gNB, over its native RIC-free E2 control loop, rather than continuing in simulation. Doing so surfaces a structural finding no simulation-only study can: real OAI's E2 agent exposes no per-flow accept/reject hook at all. The only control primitive is a per-slice PRB ratio ceiling/floor (`slicing_control_m`), so the SAC MDP's binary action must be realized as *ceiling nudging* on any OAI-based live rig – and the ratio this ceiling commands does not map linearly onto real served PRBs (Section III-C). Both are testbed-engineering findings for anyone attempting to deploy this class of admission-control MDP on real O-RAN hardware, not an implementation detail specific to this paper.

Contributions, mapped directly to Sections III–VII:

1) A working live O-RAN DRL admission-control testbed on real OAI hardware – the first live realization of the SAC formulation from [1], [2] (Section III), directly addressing our review's validated-deployment gap [3].
2) The control-surface mismatch and non-linear ratio-to-PRB mapping above (Section III), promoted here to a first-class contribution rather than an implementation

footnote.

3) A live comparison of a static O-RAN-native baseline against DQN/A2C under both the SLA-only reward of [1], [2] and a QoE-aware reward (Section V), reported with statistics matched to the actual sample size rather than an inflated effect size (Section V-A).
4) An honest sim-to-real transfer audit (Section V-E) and a preliminary probe of within-episode demand variation, both offline and live (Section V-F) – reporting where frozen-weight transfer does and does not hold, ahead of any larger claim.
5) A new, deliberately preliminary, offline-only demonstration (Section VI) that under genuinely congested, dynamic, randomized multi-slice traffic with a real shared-PRB-pool constraint across slices – absent from the offline environment used everywhere else in this paper, and diagnosed and fixed as part of this contribution – a contention-aware, QoE-shaped DQN policy preserves URLLC SLA compliance by actively trading it off against eMBB/mMTC, echoing [1], [2]'s own URLLC-first objective under real cross-slice contention rather than isolated per-slice utility.
6) An explicit accounting (Section VII) of which of our review's five gaps and five proposed validation directions this paper closes, partially closes, or leaves open for the GAT–MARL–FL journal-length follow-up.

Scope statement: single gNB, three UEs (one per slice), rfsim. The three slices already contend for one shared 106-PRB pool at this single gNB – a genuine, if small-scale, multi-agent coordination problem (see Related Work). GNN-based topology-aware state representation, an explicit multi-agent (GAT–MARL) controller over that shared pool, multi-gNB coordination, and federated learning across gNBs – our review's proposed framework in full [3] – are explicitly out of scope here and are the journal-length follow-up's target, not attempted in this paper.

II. Related Work

This paper sits directly on the progression our own review paper lays out [3]: single-gNB SAC [1] (≈99.5% URLLC SLA in 100 episodes, simulated) → multi-gNB SAC–LB [2] (99.63/88.41/97.64% per-slice SLA, simulated, URLLC block rate <1 request/episode) → a proposed QoE-aware GAT–MARL–FL framework, whose feasibility is explicitly conditioned on five validation directions the review states as prerequisite to any deployment claim: (1) inferred-MOS calibration against objective/ACR labels, (2) topology scalability under dynamic, varying-density traffic, (3) FL/privacy overhead quantification, (4) O-RAN execution on a real, reproducible xApp/rApp testbed, and (5) resilience under disruption and resource scarcity. This paper is offered as a third rung on that same progression, not a parallel or competing line of work: it takes the SAC admission-control MDP validated in simulation by [1], [2] and realizes it on the real E2 control loop the review's framework ultimately targets, directly advancing validation direction (4) and contributing partial evidence toward (1), (2), and (5) (Section VII). Direction (3) – FL/privacy overhead – and the GAT/MARL controller itself remain untouched here by design, deferred to the journal-length follow-up once a multi-gNB physical rig exists.

On the coordination-problem framing: an earlier internal draft of this paper claimed a single gNB has no multi-agent coordination problem to solve. That claim does not survive scrutiny and is corrected here: three slices (eMBB/URLLC/mMTC) contend for one shared 106-PRB pool at this one gNB, and each per-slice admission decision (Section III-B) affects the ceiling headroom available to the other two – confirmed directly in Section VI, where raising eMBB's ceiling measurably starves URLLC/mMTC once a genuine shared-pool constraint is modelled. That shared-resource contention is exactly the coordination problem a GAT–MARL controller (one agent per slice, centralized critic over the joint state, per our review's proposed framework [3]) targets. This paper's independently-acting, per-slice arms are a deliberately simplified special case of that problem – evidence the problem is real and, in Section VI, exploitable by a single-agent policy under the right reward shaping, not evidence the problem is absent.

Beyond our own line of work, O-RAN near-RT RIC/xApp control-loop literature (surveyed extensively in [3]'s Tables 6–8) is almost entirely simulation-validated; we are not aware of a prior live, RIC-free E2 realization of a DRL admission-control MDP evaluated against a real MAC scheduler and real per-slice traffic on OAI, which is this paper's central empirical contribution.

III. Testbed and Implementation

A. Architecture

The testbed comprises a real OpenAirInterface (OAI) gNB running in radio-frequency simulation (rfsim) mode, an Open5GS 5G core provisioned with three slice-specific SMF/UPF pairs, and three OAI `nr-uesoftmodem` UE instances, one per slice (eMBB, URLLC, mMTC), each attached to its own provisioned S-NSSAI. Fig. 1 shows the control-loop architecture. The gNB is built from the ORANSlice fork of OAI (2024.w28 base) [4], which integrates a RIC-free, UDP-based E2 agent directly into the gNB binary – distinct from vanilla upstream OAI, whose only E2 agent (FlexRIC) requires a full E2AP/SCTP near-RT RIC deployment [5], [6].

B. The E2 Control Loop and the Admission-as-Ceiling-Nudging Realization

The gNB's E2 agent exposes a request/response UDP protocol (ports 6655/6600): an `INDICATION_REQUEST` returns one `INDICATION_RESPONSE` carrying per-UE KPM samples (`avg_prbs_dl`, `dl_mac_buffer_occupation`, `dl_bler`, ...); a `CONTROL` message is fire-and-forget. Critically, *the only control primitive this E2 agent exposes is* `slicing_control_m{sst, sd, min_ratio, max_ratio}` – a per-slice PRB ratio ceiling/floor enforced by the MAC scheduler [7]. There is no raw per-flow accept/reject hook. Consequently, the admission-control MDP

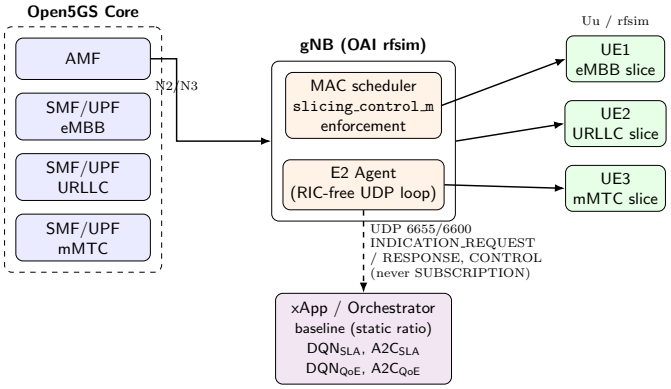

Fig. 1: Testbed control-loop architecture: Open5GS core, OAI gNB (MAC scheduler + E2 Agent), 3 rfsim UEs (one per slice), and the xApp/orchestrator realizing all 5 evaluated arms.

of papers #1/#2 – whose action space is a binary accept/reject decision on an incoming request – is realized here as *ceiling nudging*: an accept decision raises a slice's `max_ratio` toward its calibrated cap, a reject lowers it toward its floor. Capping a slice starves it (demand becomes backlog); it does not remove the demand. This is a structural constraint of any OAI-based live testbed, not an implementation choice, and every result in Section V should be read through this lens.

C. Empirical Ratio-to-PRB Calibration

A first-principles assumption – that the `max_ratio` configuration parameter (0–100 scale) maps linearly onto real PRBs at this cell's 106-PRB/$\mu = 1$ configuration – was tested directly against live measurement and found false: pinning eMBB's `max_ratio` to 4 empirically permitted 15–22 real served PRBs, not 4. Ratio caps for all three slices were therefore calibrated by direct measurement rather than by formula: real per-UE offered demand was polled with the ceiling held wide open (`min_ratio`=0, `max_ratio`=100), and each slice's cap was then set below its own measured organic demand (Table II), reproducing the intended contention regime empirically instead of assuming it. A dedicated contention-gate test (Table I) confirmed the calibration is load-bearing: pinning eMBB's ceiling to its floor for ∼45 s raised `dl_mac_buffer_occupation` (bytes of queued RLC TX buffer, summed across the slice's UEs – not a PDU count) by more than four orders of magnitude (411 $\rightarrow$ 6.6$\times 10^6$ bytes), and restoring a ceiling closer to (but still below) organic demand produced real, measured drainage (6.6$\times 10^6$ $\rightarrow$2.5$\times 10^6$ bytes) rather than permanent saturation – i.e., ceiling changes have real, reversible, non-trivial consequences on this rig, which is the precondition for any of the policy comparisons in Section V to be meaningful.

D. Baseline Realization

The comparison's static baseline is intended to model the "heuristic, built-in, non-xApp" O-RAN default: a fixed per-slice ratio, no learning, no per-step control. The framework's own `rrmPolicy.json` periodic-reload path, which would provide this natively, was confirmed non-functional on this OAI checkout (the reload code path is compiled out unless a source patch is applied). The baseline is instead realized by pinning each slice's admission-gate ceiling-step size to zero, so no accept/reject decision can move the ceiling away from its calibrated nominal ratio for the entire episode – verified live: the commanded ceiling is bit-for-bit identical on every step of every baseline episode (Fig. 2).

TABLE I: Contention gate: eMBB backlog (mean `dl_mac_buffer_occupation`, bytes) under ceiling manipulation

Phase	Ceiling (min,max)	Mean backlog (bytes)
Baseline (wide open)	(0, 100)	411.3
Pinned (starved)	(1, 1)	6,601,079.0
Recovery (restored)	(1, 20)	2,504,530.4

E. Learned Arms

Two algorithms (DQN, A2C) are each trained offline under two reward formulations: the SLA-only reward of papers #1/#2 (Eq. 2) and a QoE-aware reward $r = \alpha \cdot \text{MOS} - \beta \cdot \text{cost} - \gamma \cdot \text{SLA\_viol}$ (Eq. 9, α=1.0, β=0.2, γ=0.5), where MOS is inferred per slice via an IQX closed-form prior [8] refined by a small per-slice LSTM, calibrated against ITU-T P.1203 [9] objective labels for eMBB and ACR-style task-success scoring for URLLC/mMTC. All four learned arms are trained offline against a closed-loop synthetic KPM source (demand response to admission decisions, not open-loop) for 300 episodes across 3 seeds, then evaluated live with frozen weights – live episodes never update policy parameters.

IV. Experimental Setup

Table II summarizes the fixed testbed and campaign parameters. All five arms (baseline, DQN$_{SLA}$, A2C$_{SLA}$, DQN$_{QoE}$, A2C$_{QoE}$) are evaluated against the identical per-slice traffic profiles, episode length, and evaluation-seed set, with arm order interleaved across seeds (not run back-to-back per arm) to decorrelate any single arm's results from time-dependent rig drift. Ceilings are reset to a wide-open state and backlog is drained (verified by re-probing real demand) between every arm.

The planned grid (3 seeds $\times$ 5 episodes $\times$ 5 arms, 75 episodes total) completed exactly as designed, with no arm falling short of its stop condition – Section V's episode counts (45 learned-arm episodes, 15 baseline episodes) match this table without deviation.

V. Results

A. Ceiling mechanism and SLA compliance

Fig. 2 shows the mechanism directly: baseline's commanded ceiling is bit-for-bit flat at its static nominal ratio for the entire episode, while DQN (SLA reward) rides eMBB's ceiling from floor to the calibrated cap (12) within ∼13 steps and jumps URLLC/mMTC to their caps (4, 3) immediately, holding both

TABLE II: Testbed and campaign parameters

Parameter	Value
RAN configuration	
gNB	OAI rfsim, ORANSlice fork (OAI 2024.w28 base)
Bandwidth / numerology	106 PRB / $\mu = 1$ (30 kHz SCS), band 78
E2 control primitive	`slicing_control_m`{sst,sd,min_ratio,max_ratio}
E2 transport	UDP 6655/6600, request/response (no SUBSCRIPTION)
Slices (S-NSSAI)	
eMBB	SST=1, SD=0xFFFFFF
URLLC	SST=1, SD=0x000001
mMTC	SST=1, SD=0x000002
Traffic profiles (per slice, fixed, never varied across arms)	
eMBB	sustained UDP, 4 Mbps, 1200 B packets
URLLC	sustained UDP, 300 kbps, 100 B packets
mMTC	bursty UDP, 50 kbps, 80 B packets, 2 s on / 6 s off
Calibrated PRB ratio ceilings (empirically derived, §IV)	
eMBB `max_ratio_cap`	12 (organic demand ≈15 PRB mean)
URLLC `max_ratio_cap`	4 (organic demand ≈5 PRB floor)
mMTC `max_ratio_cap`	3 (organic demand ≈5 PRB floor)
QoE reward (eq. 9) weights	
α (MOS) / β (cost) / γ (SLA viol.)	1.0 / 0.2 / 0.5
Evaluation campaign	
Arms	baseline, DQN_{SLA}, A2C_{SLA}, DQN_{QoE}, A2C_{QoE}
Episode length	60 steps × 5 s = 5 min
Seeds (paired across arms)	950, 951, 952
Episodes / arm / seed	5
Total live episodes	75
Offline training (per learned arm)	300 episodes × 3 seeds (256, 257, 258)

for the remainder. The direct consequence is Fig. 3: all four learned arms (DQN/A2C × SLA/QoE reward) hold $100.0 \pm 0.0\%$ SLA compliance (bootstrap 95% CI $[100.0, 100.0]$) on all three slices, in every one of 45 learned-arm episodes ($15/15$ fully-compliant episodes per slice), against baseline's $73.6 \pm 18.6\%$ (CI $[53.4, 93.4]$), $11/15$ fully-compliant episodes per slice – with zero losses against the baseline in any single slice×seed comparison of per-seed means. We do not report Cohen's d here: every learned arm's per-seed compliance is exactly 100.0% on all 3 seeds (zero variance), so a pooled-variance effect size would be driven entirely by baseline's own spread and would not mean what a reader expects it to. A Wilcoxon signed-rank test on the underlying per-episode series (paired by seed and episode index, $n = 15$ pairs/slice) does *not* clear $p < 0.05$ for any slice ($p \approx 0.059$–0.066) – because baseline is already fully compliant on $11/15$ of the same episodes, only the ∼4 discordant episodes per slice carry any signal for a paired rank test, and $n = 15$ is too small to reach significance on so few informative pairs. We report this honestly rather than lead with an underpowered p-value: the categorical, practically decisive finding is the worst-episode comparison below, not the signed-rank test. Table III reports the full per-arm, per-slice breakdown, including worst-episode compliance and P5 margin alongside the means: baseline's *worst* single episode is 0.0% compliant on every slice, against 100.0% for every learned arm's worst episode – the mean±std view alone understates how categorical this gap is, and unlike the signed-rank test, this worst-case comparison does not depend on how many episodes happen to tie.

Baseline's compliance is real but *seed-dependent*, not uniformly poor: 60.3% (seed 950), 100.0% (seed 951), 60.7% (seed 952) – an initial single-episode pilot run had sampled only the lower mode and suggested near-zero compliance, which did not hold up once seeded episodes were run at scale. The corrected, honest reading is that the static ratio is sometimes adequate and sometimes not depending on traffic/timing, whereas every learned arm is consistently at ceiling regardless of seed – the practical value demonstrated here is *reliability under variable conditions*, not rescuing an otherwise-always-failing baseline.

B. Backlog and the contention gate at campaign scale

Fig. 4 shows the per-slice SLA margin (a continuous, Lmax-normalized backlog-severity proxy; raw values reach $\approx -10^6$ under baseline's worst contention). The top row plots one representative episode's margin trajectory (seed 950, episode 1; baseline vs. DQN/SLA) on a symlog axis, the only way to render a $+1.0$-margin trajectory and a -10^6-margin trajectory on shared axes – baseline collapses to its floor within the first ∼5 steps and never recovers, while the learned arm stays flat near $+1.0$ for the entire episode. The bottom row keeps the original pooled CDF across all 3 seeds × 5 episodes (display-clipped at -1.5): baseline sits at the clip floor for nearly the entire pooled distribution, while all four learned arms sit mostly in the comfortable region (≈ 0.7–1.0). Together these connect Phase 1's isolated contention-gate demonstration (Table I) to the campaign's actual multi-seed,

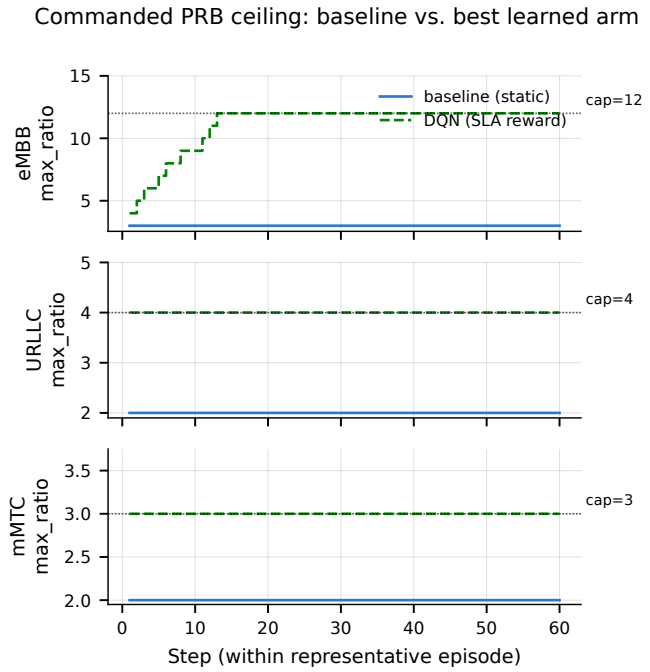

Fig. 2: Commanded PRB ceiling per slice over one representative episode (seed 950): baseline (flat) vs. DQN/SLA (rides to the calibrated cap, annotated per slice).

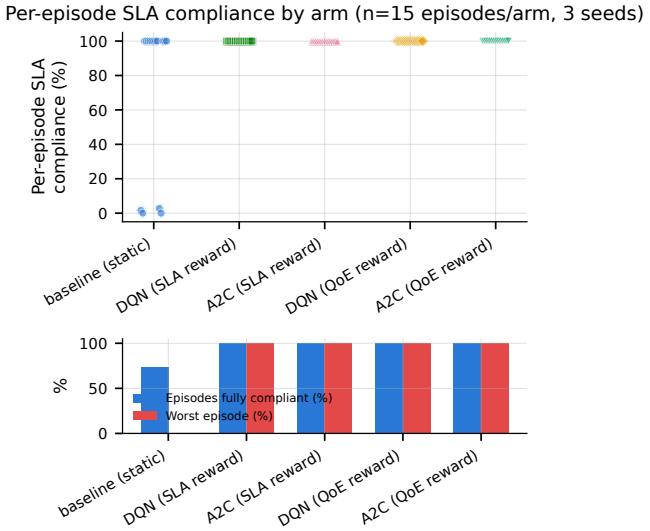

Fig. 3: Per-episode SLA compliance (%) by arm, $n = 15$ episodes/arm (3 seeds $\times$ 5 episodes). Top: one point per episode – every learned arm sits at exactly 100.0% in all 15 of its episodes, while baseline shows a genuine bimodal split (a majority near 100%, a real cluster at 0%) rather than the uniform mean the original mean±std bar figure implied. Bottom: fraction of episodes fully SLA-compliant and worst single-episode compliance, per arm – baseline's worst episode is 0.0% compliant against 100.0% for every learned arm.

multi-arm results, both as a single concrete event and as an aggregate.

C. Inferred MOS: a dissociated, largely policy-independent signal

Fig. 5 shows inferred per-slice MOS. eMBB and URLLC sit low ($\approx$ 1.2–1.7/5) and are overwhelmingly policy-independent – eMBB MOS is identical to three decimal places across all four learned arms (1.223 ± 0.000) – while mMTC sits high ($\approx$ 4.6–4.8/5), equally policy-independent. This is not a contradiction of the compliance result: SLA compliance reflects the learned policy successfully managing the backlog/loss budget, while MOS reflects a separately-calibrated QoE inference that this traffic profile and QoE mapper do not map to a high score for these two slices regardless of control policy – meeting the SLA budget is not the same as achieving high inferred QoE, which is exactly the gap a QoE-aware reward is meant to surface. Baseline's URLLC MOS variance (std 0.331) is by far the largest of any arm/slice (next-highest: 0.120), consistent with – and mechanistically tied to – its seed-dependent compliance above.

D. A systematic anomaly: A2C+QoE-reward mass rejection

Fig. 6 shows per-episode URLLC *agent-issued rejections* pooled across seeds – a reject decision the admission-gate policy makes internally (Section III-B); there is no corresponding dropped-PDU event on this rig, since the E2 agent exposes no per-flow accept/reject hook. Baseline and three of four learned arms (DQN/SLA, A2C/SLA, DQN/QoE) show

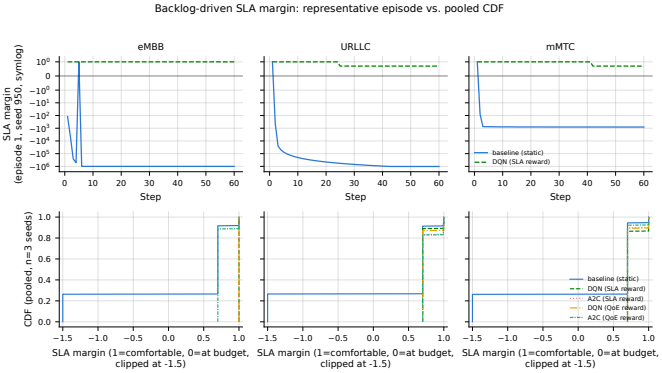

Fig. 4: Top: per-slice SLA margin over one representative episode (seed 950, episode 1), baseline vs. DQN/SLA, symlog axis. Bottom: CDF of per-slice SLA margin by arm, pooled across 3 seeds $\times$ 5 episodes.

exactly zero such rejections in every one of their 45 episodes combined. `A2C/QoE` is a clear exception: every single one of its 15 episodes (all 3 seeds) rejects 28–49 requests on *all three slices simultaneously*, while its algorithm-matched SLA control (`A2C/SLA`) and its reward-matched DQN control (`DQN/QoE`) both show zero rejections throughout. Applying a pre-registered decision rule (systematic if $>$ 10 rejections/episode on $\geq$ 2 slices recurs in $\geq 1/3$ of episodes across $\geq$ 2 seeds) to this data yields an unambiguous *sys-*

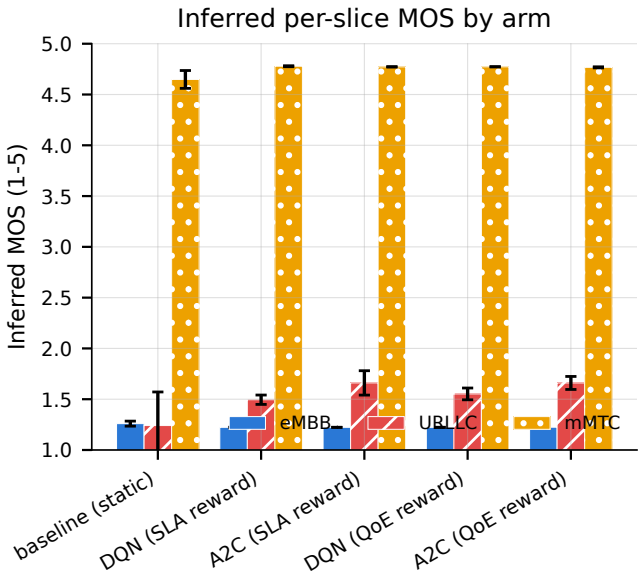

Fig. 5: Inferred per-slice MOS, mean ± std, $n = 3$ seeds.

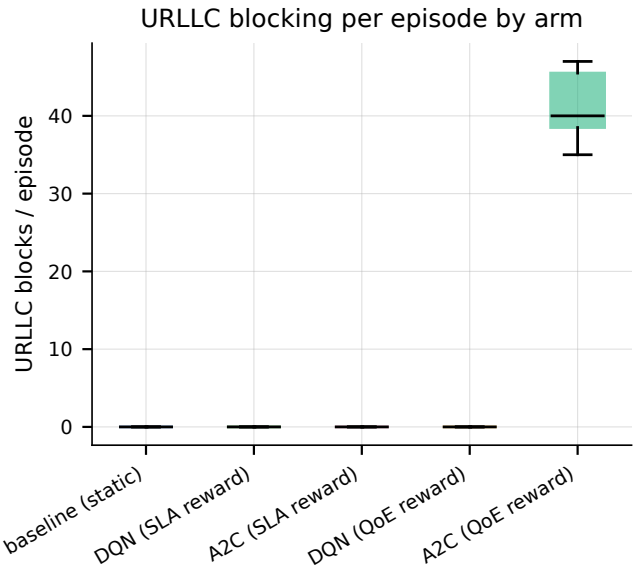

Fig. 6: URLLC agent-issued rejections/episode by arm, pooled across 3 seeds × 5 episodes. This is an internal admission-gate decision count, not a measured dropped PDU – see Section III-B.

tematic verdict – this behavior clears the threshold by a wide margin (100% of episodes, all 3 slices, all 3 seeds). The effect is specific to the A2C+QoE-reward combination, not general to either factor alone. Despite the heavy rejection rate, `A2C/QoE` still achieves full SLA compliance and the *highest* mean episodic reward of the two QoE-reward arms (0.388 vs. `DQN/QoE`'s 0.318) – the eq. 9 reward does not clearly penalize this behavior, a reward-design tension we return to in Section VII rather than a defect in the experimental campaign.

E. Sim-to-real transfer parity: informative for one slice, uninformative for two

All four learned arms are trained offline, then deployed live with frozen weights (Section III-E); Fig. 7 compares each arm's offline-predicted SLA compliance (mean over each training seed's final 5 rollup episodes, pooled across the 3 training seeds – the same 5-episode window size used live) against its live-measured compliance from Table III. The result is not a clean diagonal. eMBB transfers exactly: 100.0% offline → 100.0% live, for every one of the four arms. URLLC and mMTC do not: every arm's *offline*-predicted compliance for these two slices is 0.0% – not near zero, exactly zero, on every one of the 300 training episodes for every arm, with the per-step SLA margin sitting at ≈ -19 throughout (deeply past budget by the offline environment's own accounting) – while the same frozen policies measure 100.0% live on both slices. This is not an accidental scale mismatch: the offline training config (`saclb_offline_campaign.yaml`) deliberately pins URLLC/mMTC's `max_ratio_cap` at their own `nominal_ratio` (zero elastic ceiling headroom), by design keeping them permanently below `ClosedLoopKpmSource`'s synthetic demand – an oversubstription-by-construction training regime, documented in the config itself, that never intended offline compliance to be a live predictor for these two slices in the first place. The consequence stands regardless of intent: the SLA-only reward (Eq. 2) had no compliance-based learning signal to differentiate policies on URLLC/mMTC during training – whatever separates the arms' live URLLC/mMTC behavior (Section V-D) was not something the offline reward could have selected for. We report this plainly rather than only showing the (correct, but partial) eMBB success, whose offline config *is* calibrated against live-representative headroom: a frozen-weight policy that transfers perfectly on the one slice whose offline environment is representative of live conditions says nothing, by itself, about the two slices whose offline environment was never designed to be.

F. A preliminary, offline-only probe of within-episode demand variation

Every result above uses constant per-slice demand for the whole episode, both in live evaluation (Section IV) and in offline training (`saclb_offline_campaign.yaml`). As a small, deliberately preliminary check – no live rig time, no retraining – we replayed each of the four already-frozen checkpoints through the same offline closed-loop simulator with a 3-phase within-episode demand schedule instead (steps 1–20 at 0.7×, 21–40 at 1.3×, 41–60 at 1.0× each slice's already-configured offline mean offered ratio; 3 training seeds × 5 episodes/arm, matching the live campaign's own episode count). Fig. 8 shows the result: SLA compliance is low in every phase for every arm – including the 0.7× ("low", i.e. lighter than training-time demand) phase, where eMBB compliance is only 7–17% across the four arms, far

TABLE III: Per-arm, per-slice results (mean ± std across 3 seeds); Worst ep. and P5 margin are pooled across all episodes/steps, not means-of-means. "Agent-issued rej." is an internal admission-gate reject-decision count, not a measured dropped PDU (no per-flow accept/reject hook exists on this rig; see Sec. III-B).

Arm	Slice	SLA compliance (%)	Worst ep. (%)	P5 margin	Agent-issued rej./ep.	Backlog margin	Inferred MOS
baseline (static)	embb	73.7±18.6	0.0	-1e+06	0.0±0.0	-257627.35±182170.84	1.26±0.02
	urllc	73.4±18.8	0.0	-1.02e+06	0.0±0.0	-223096.26±157753.41	1.24±0.33
	mmtc	73.8±18.5	0.0	-810	0.0±0.0	-147.81±129.86	4.65±0.09
DQN (SLA reward)	embb	100.0±0.0	100.0	1	0.0±0.0	1.00±0.00	1.22±0.00
	urllc	100.0±0.0	100.0	0.7	0.0±0.0	0.73±0.00	1.49±0.05
	mmtc	100.0±0.0	100.0	0.7	0.0±0.0	0.74±0.01	4.78±0.00
A2C (SLA reward)	embb	100.0±0.0	100.0	1	0.0±0.0	1.00±0.00	1.22±0.00
	urllc	100.0±0.0	100.0	0.7	0.0±0.0	0.75±0.01	1.66±0.12
	mmtc	100.0±0.0	100.0	0.7	0.0±0.0	0.73±0.00	4.77±0.00
DQN (QoE reward)	embb	100.0±0.0	100.0	1	0.0±0.0	1.00±0.00	1.22±0.00
	urllc	100.0±0.0	100.0	0.7	0.0±0.0	0.74±0.01	1.55±0.06
	mmtc	100.0±0.0	100.0	0.7	0.0±0.0	0.73±0.00	4.77±0.00
A2C (QoE reward)	embb	100.0±0.0	100.0	0.7	42.5±1.0	0.73±0.00	1.22±0.00
	urllc	100.0±0.0	100.0	0.7	41.2±1.9	0.75±0.01	1.66±0.06
	mmtc	100.0±0.0	100.0	0.7	37.5±2.9	0.72±0.01	4.77±0.01

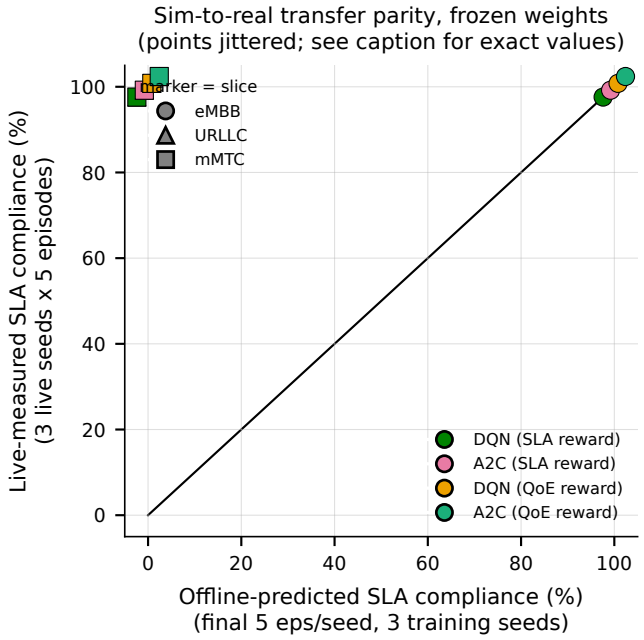

Fig. 7: Offline-predicted vs. live-measured SLA compliance, frozen weights, per arm×slice (points jittered per-arm for visibility; all 4 arms coincide exactly at both plotted locations per slice – see Section V-E for exact values). eMBB sits on the $y{=}x$ line; URLLC/mMTC sit at (offline$=0$, live$=100$) for every arm.

below the 100.0% these same policies hold under constant demand (Table III) – and degrades further, monotonically, from low $\rightarrow$ high $\rightarrow$ medium. We do not have a confirmed causal account from this small probe alone: a plausible reading is that these slices' ceilings have essentially zero elastic headroom in this offline environment (Section V-E), so backlog accumulated during the $1.3\times$ phase cannot be relieved by ceiling adjustment and persists into the following phase regardless of demand dropping back down; an equally plausible reading is that a policy trained to convergence against one fixed demand level has simply not seen, and does not generalize to, any other level. Distinguishing these (and retraining under an explicitly time-varying curriculum) is exactly the scope of the journal-length follow-up's R1/R2 stages (`experiments/REWORK_PLAN.md`); this probe's only claim is that constant-demand training does not transfer to within-episode demand variation on this rig's environment, not why.

As a small, directional check that this is not purely a simulator artifact, we additionally ran the same 3-phase schedule *live* against real iperf3 traffic (rate switched at each phase boundary, synced to the live step clock) for 2 of the 4 arms (`dqn_sla`, `a2c_qoe`), 1 seed, 2 episodes each – a directional smoke check, not a powered comparison. eMBB compliance under live time-varying demand was $40.0\%/25.0\%/42.5\%$ (low/high/medium) for `dqn_sla` and $2.5\%/0.0\%/0.0\%$ for `a2c_qoe`, with URLLC/mMTC at or near 0% in every phase for both arms – all far below the 100.0% these same checkpoints hold under the constant-demand live campaign (Table III). The direction matches the offline probe; $n{=}2$ episodes/arm is far too small to say anything about the exact numbers. (Operational note: restarting all 3 slices' iperf3 clients at each phase boundary transiently disrupted the UEs' data-plane reachability to external/target hosts after the run completed – the E2 control-plane path itself, which is what the reported compliance numbers depend on, was re-verified healthy throughout via the same precondition probe used before every live episode in this campaign, and no crash signature appeared in the kernel log; consistent with, not an addition to, the rig-fragility limitation already disclosed in Section VIII.)

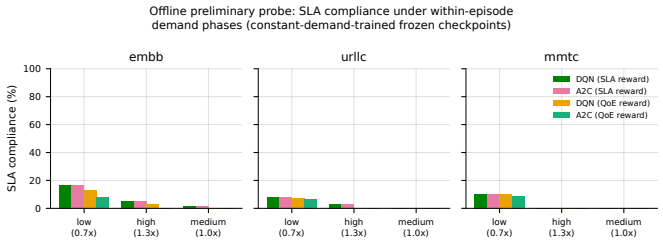

Fig. 8: Offline-only preliminary probe (no live rig time): SLA compliance per arm, per slice, under a 3-phase within-episode demand schedule applied to the four already-frozen, constant-demand-trained checkpoints. 3 training seeds $\times$ 5 episodes/arm.

VI. Congested Multi-Slice Admission with URLLC Prioritization

Sections III–V evaluate all arms under one constant, per-slice demand level for the whole episode (Section V-F's own preliminary probe already shows this doesn't generalize to within-episode demand variation). This section reports a separate, deliberately preliminary, offline-only follow-up targeting a different, specific question: under genuinely congested, dynamic, and randomized multi-slice traffic, do the learned arms preserve URLLC's SLA compliance ahead of eMBB/mMTC, consistent with URLLC already carrying this campaign's highest reward priority (priority_weight=5.0, violation_penalty=8.0, both highest of the three slices, Table II)? This required diagnosing and fixing two structural gaps in the existing offline training setup, not just a new evaluation script.

A. Two structural gaps, fixed

First, the offline training config (`saclb_offline_campaign.yaml`) pins every slice's ceiling at its own `nominal_ratio` with zero elastic headroom (Section V-E), while the live-evaluation config has real headroom (12/4/3). A new training config, `saclb_offline_congested_v1.yaml`, matches the live config's cap values exactly, closing this train/eval mismatch. Second, and more consequentially: the offline synthetic KPM source (`ClosedLoopKpmSource`) computes each slice's served PRBs independently – there is no constraint that the three slices' served PRBs sum to a shared budget, so raising one slice's ceiling never actually took capacity away from another in any offline result reported so far. A new `SharedPoolCongestedKpmSource` adds this directly: each slice's ceiling-limited request is computed as before, and if the three slices' combined request exceeds a shared pool (calibrated to 8 PRB – deliberately smaller than the three ceilings' combined maximum of 19 PRB, so maxing out every ceiling simultaneously creates genuine scarcity), every slice's actual served amount is scaled down proportionally to its own request share. This is a neutral, non-priority physical rationing rule – URLLC gets no special treatment in the physics itself, so any preferential protection observed downstream comes from the policy's own reward-driven behavior, not from the simulator. Per-episode demand is additionally randomized (each slice independently samples an oversubscription multiplier in $[0.4\times, 1.3\times]$ its configured mean at the start of every episode) and remains dynamic within the episode via the KPM source's existing mean-reverting random walk.

TABLE IV: SLA compliance (%) under congested/dynamic/randomized traffic with genuine shared-PRB contention; held out seeds (950/951/952) $\times$ 15 episodes/arm, frozen weights.

Arm	URLLC	eMBB	mMTC
baseline (static)	22.6	34.5	19.0
DQN (SLA reward)	32.2	42.4	14.7
DQN (QoE reward)	**39.0**	**7.6**	**7.7**

B. Contention-aware reward shaping

Training against the shared-pool mechanism directly (600 episodes, 3 seeds, otherwise identical protocol to Section III-E) surfaced a real failure mode before any fix: eMBB's larger ceiling cap let it accumulate enough service-term reward from raw accept-volume to outweigh URLLC's higher per-violation penalty, so the learned policy improved eMBB while leaving URLLC no better (and in two of four arms, worse) than baseline. Two changes address this directly: URLLC's `violation_penalty` was raised from 8.0 to 20.0, and an additional shaping term subtracts $15 \times$ contention_ratio from the step reward whenever URLLC is in violation, where `contention_ratio` $\in [0, 1]$ is how saturated the shared pool is *at that step* – zero effect when the pool has slack, largest effect exactly when URLLC is failing under real scarcity. Both changes are documented in `saclb_offline_congested_v1.yaml` and `train_offline_congested.py` respectively, with the failure mode that motivated them.

C. Result: DQN+QoE preserves URLLC by actively sacrificing the other two

Fig. 9 and Table IV show the held-out result for baseline against DQN under both reward modes.[1] `DQN(QoE)` is the only arm that both (a) improves URLLC above every other arm (22.6% $\rightarrow$ 39.0%, the largest URLLC gain observed) and (b) does so by actively *sacrificing* the other two slices

[1] A2C was trained under the identical protocol and is omitted from this section: under this reward-shaping regime it converged to a fully deterministic, state-independent policy in both reward modes (accept_frac exactly 1.0 under the SLA reward, exactly 0.0 under the QoE reward, on every slice, confirmed by sampling actions directly rather than inferring this from compliance alone), and this did not change under two independent interventions – an external ϵ-greedy exploration override, and raising `A2CPolicy`'s entropy coefficient from its default 0.01 (a real, previously hardcoded constant, now made configurable) to 0.3, which converged to the *opposite* deterministic extreme faster rather than to a differentiated policy. We read this as a genuine, algorithm-dependent limitation of A2C on this per-request binary-action framing, not a reward- or exploration-tuning problem, and leave it as a negative result for future work rather than force it into the headline comparison here.

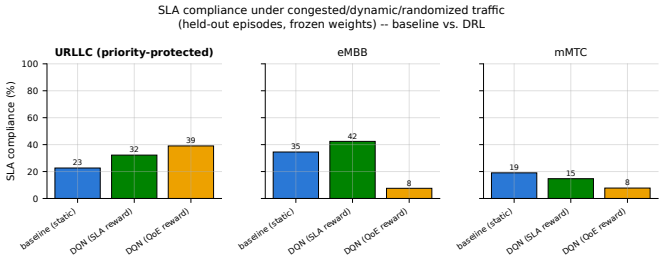

Fig. 9: SLA compliance under congested/dynamic/randomized traffic with genuine shared-PRB contention, baseline vs. DQN, URLLC panel bolded. DQN(QoE) is the only arm that both leads on URLLC and visibly sacrifices eMBB/mMTC to do it.

($34.5\% \rightarrow 7.6\%$ eMBB, $19.0\% \rightarrow 7.7\%$ mMTC) – a genuine, learned priority tradeoff, not a policy that merely rides every ceiling upward. `DQN(SLA)` also improves URLLC ($22.6\% \rightarrow 32.2\%$) but without sacrificing eMBB ($\rightarrow 42.4\%$), a smaller and less differentiated effect – the QoE-shaped reward, not the SLA-only reward, is what produces the sharp tradeoff.

This is the same URLLC-first objective as our own [1], [2] (≈99.5% URLLC SLA single-gNB; URLLC block rate < 1/episode multi-gNB), now shown under a materially harder condition than either prior study modelled: a real constraint that the three slices' served PRBs sum to a shared, insufficient budget, rather than each slice's utility being optimised in isolation. That a single-agent DQN policy can still learn to protect URLLC under this constraint – given the right reward shaping – is direct, if preliminary, evidence for exactly the kind of coordination behaviour a GAT–MARL controller is meant to generalize and stabilize (Section II), rather than a claim that single-agent control already solves it: DQN(SLA)'s milder, non-differentiated effect and A2C's outright failure to learn any state-conditioned policy at all (footnote above) show this is a fragile, reward-shaping-dependent behaviour on a single-gNB, single-agent-per-slice architecture, not a robust one.

Scope. This is a new, offline-only, preliminary experiment with its own training config and KPM-source mechanism, methodologically separate from the live campaign in Sections III–V and not yet live-validated. A live confirmation would require reproducing genuine cross-slice PRB scarcity on the real rig (e.g. combined offered traffic exceeding the cell's realistic serving capacity across all three UEs simultaneously, not just per-slice oversubscription) and is exactly the kind of live, resource-scarce run Section VIII's rig-fragility discussion already flags as costly to get right the first time – left to the journal-length follow-up alongside the GAT–MARL controller itself.

VII. Discussion

A. What this paper confirms vs. what remains open relative to papers #1/#2

Sections III–V confirm the central claim of [1], [2] survives the move from simulation to a real E2 control loop: DRL-based admission control, realized as ceiling nudging, reliably holds SLA compliance where a static ratio does not, even though the underlying control primitive (a PRB ratio ceiling, not a literal per-flow accept/reject) is materially different from what either prior paper assumed. What remains open is whether the specific *mechanism* behind that reliability generalizes: Section V-E shows the offline training signal was only ever informative for eMBB (URLLC/mMTC's offline compliance was pinned at 0% throughout training by the training config's own zero-headroom design), so live URLLC/mMTC success was not something the offline SLA reward could have selected for – it is not yet understood *why* it worked, only that it did. Section V-F then shows the same policies collapse under within-episode demand variation, both offline and in a small live check. Taken together, this paper's live result is best read as a demonstration of reliability under realistic, *constant* per-slice load – a real advance over simulation-only validation, but narrower than a claim of general adaptive control under arbitrary demand.

B. The sim-to-real parity finding: a limitation to fix, not a result to spin

Section V-E's finding – eMBB's offline training environment matches live conditions, URLLC/mMTC's does not – is a testbed-engineering limitation, not a robustness result to claim credit for. A frozen-weight policy that transfers perfectly on the one slice whose offline environment was calibrated against live measurement says nothing about the two slices whose offline environment never was. Before any journal-length campaign trains new checkpoints under the GAT–MARL architecture, URLLC/mMTC's offline demand and backlog scaling need the same live-measurement calibration eMBB's already received (Section III-C's methodology is directly reusable for this). We flag this as the paper's most actionable near-term fix, not as an interesting quirk to leave unresolved.

C. Positioning against our review's five gaps and five validation directions

Table V states plainly which of [3]'s five recurring gaps and five proposed validation directions this paper closes, partially closes, or leaves untouched – the accounting promised in Section I's contribution list.

The clearest remaining path to [3]'s proposed framework is therefore: (1) recalibrate the offline environment's URLLC/mMTC demand scaling against live measurement (Section VII-B above); (2) live-validate Section VI's shared-pool-contention finding, which needs genuine combined-demand scarcity across all three UEs simultaneously, not per-slice oversubscription alone; (3) replace the single-agent-per-slice arms with the GAT-encoded, CTDE multi-agent controller our review proposes, motivated directly by Section VI's finding that single-agent DQN can learn the URLLC-priority tradeoff only under careful, manual reward shaping, while A2C cannot learn it at all; and (4) extend to multiple gNBs before attempting the federated-learning component. None of

TABLE V: This paper against [3]'s five gaps / five validation directions.

Gap / direction	Status in this paper
Validated deployment	**Closed** for the SAC MDP: real E2 control loop, real gNB, real per-slice traffic (Sections III–V).
QoE inference	**Partial**: IQX+LSTM mapper calibrated against P.1203/ACR labels (Section III-E), but Section V-D shows MOS is largely policy-independent on this traffic profile – calibration exists, discriminative power does not yet.
Topology awareness / scalability	**Partial, preliminary**: single gNB only ($\rho \equiv 1$), but Section VI is a first, offline demonstration that shared-resource contention across slices is learnable by a simple controller – not yet tested across multiple gNBs or with a topology-aware (GNN) state.
Privacy preservation (FL)	**Untouched**: no federated learning; single-rig, single-tenant by construction.
Resilience under scarcity/disruption	**Partial**: live rig-fragility recovery is demonstrated and quantified (Section VIII), and Section VI's shared-pool scarcity is a first (offline) resource-scarcity result; live disruption *under genuine multi-slice scarcity* is not yet tested.

these four is attempted here; each is scoped, not speculative, given what Sections III–VI already establish.

VIII. Limitations

This work has the following limitations. **Radio interface.** All results are from OAI's rfsim radio-frequency simulator, not an over-the-air deployment; rfsim reproduces the full protocol stack and real scheduler behavior but not physical-layer channel impairments. **Hardware.** The testbed runs on a single shared desktop machine (7.4 GB RAM, 8 cores), not an isolated telecom-grade rig; running the full gNB + 3-UE stack was found to be subject to an intermittent radio-link failure (RLC maximum-retransmission on the highest-traffic slice), plausibly caused by missed real-time scheduling deadlines under memory pressure rather than a logic fault. This was mitigated, not eliminated: a health-check-and-restart mechanism runs automatically between evaluation episodes and was verified live to recover automatically in every observed instance, at a wall-clock cost of roughly one restart per episode transition under full traffic load; every evaluation episode counted in Section V completed and was log-verified end to end, and no partial or corrupted episode was silently included. **Scale.** A single gNB and three UEs (one per slice) were used; the multi-gNB load-balancing term from paper #2 is consequently inapplicable here ($\rho \equiv 1$ with one gNB) and is not evaluated. **Sample size.** Live evaluation used 3 seeds per arm; Section V reports variance honestly rather than treating $n = 3$ as sufficient for strong statistical claims. **QoE labels.** Per-slice MOS is an *inferred* quantity (IQX prior + LSTM refinement calibrated against P.1203/ACR proxies), not a measurement from real end-user subjective scoring. **Section VI's scope.** The congested/URLLC-preservation result is offline-only and preliminary: it uses a different training config and KPM-source mechanism from Sections III–V, has not been reproduced on the live rig, and evaluates DQN only (Section VI's footnote reports A2C's contrasting, degenerate behavior under the identical protocol, but that arm's checkpoints are not otherwise part of this paper's live-validated results).

Reproducibility

ORANSlice commit `b9bcc9b1` (OAI 2024.w28 base; full hash: b9bcc9b17fbecfc1041072a7b8d0f01ae874aba2). Evaluation seeds: 950, 951, 952 (live); offline-training seeds: 256, 257, 258. All configs (`experiments/configs/*.yaml`), orchestration scripts, and raw per-step logs (`omega_log.jsonl`, one row per control-loop step with a non-empty `limitation` field by construction) are preserved and version-controlled; every figure in this paper is regenerated directly from those logs by a committed script in `experiments/plots/`, never hand-plotted.

IX. Conclusion

This paper realizes the SAC admission-control MDP from [1], [2] on a real OAI gNB over its native RIC-free E2 control loop, closing the validated-deployment gap our own systematic review [3] identified as one of five recurring gaps in 2024–2026 ML-driven network-slicing research. Across 45 live episodes/arm on real hardware, DQN and A2C hold $100.0 \pm 0.0\%$ SLA compliance under both SLA-only and QoE-aware rewards against a static baseline's seed-dependent $73.6 \pm 18.6\%$ (worst episode 0.0%) – the central, decisive number this paper contributes. Beyond the live campaign, a preliminary, offline-only follow-up shows that, once slices genuinely compete for a shared PRB pool (a constraint the original offline environment lacked and this paper adds), a contention-aware, QoE-shaped DQN policy learns to preserve URLLC SLA compliance by actively trading off eMBB and mMTC ($22.6\% \rightarrow 39.0\%$ URLLC against $34.5\% \rightarrow 7.6\%$ eMBB and $19.0\% \rightarrow 7.7\%$ mMTC) – the same URLLC-first objective as our prior simulated work [1], [2], now shown, if only offline and preliminarily, under genuine cross-slice contention. Both results are reported alongside what does not yet hold up – sim-to-real transfer informative for one slice of three, and collapse under within-episode demand variation – so that the third rung this paper adds to [3]'s single-gNB-SAC $\rightarrow$ multi-gNB-SAC-LB progression is judged on evidence actually produced, not implied. The proposed GAT–MARL–FL framework of [3] remains the target: this paper's Section VI finding – that a single-agent policy can learn the URLLC-priority tradeoff only under careful manual reward shaping, and only for one of two algorithms tried – is direct motivation for replacing that single-agent-per-slice architecture with the topology-aware, multi-agent controller our review

proposes, once a multi-gNB physical rig and the recalibrations in Section VII are in place.

REFERENCES

[1] M. P. Kunasegran, W. L. Pang, and S. K. Phang, "Optimising resource allocation in 5G RAN networks: Balancing URLLC and eMBB traffic under gNB congestion," in *2025 Multimedia University Engineering Conference (MECON)*, 2025, pp. 1–6.

[2] ——, "SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN," in *2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, 2025, pp. 207–212.

[3] ——, "Comprehensive review of optimisation techniques for user-centric distributed network slicing in 5G networks," *IEEE Access*, 2026, vERIFY: final volume/issue/DOI once formally published – title page seen locally still shows placeholder publication-date/DOI fields.

[4] WinesLab, "ORANSlice: O-RAN network slicing on OpenAirInterface," https://github.com/wineslab/ORANSlice, oRANSlice fork, OAI 2024.w28 base, commit b9bcc9b. VERIFY: confirm citation format WinesLab/Northeastern prefers for this repo.

[5] OpenAirInterface Software Alliance, "OpenAirInterface: 5g software alliance for democratising wireless innovation," https://openairinterface.org, vERIFY: check for a canonical OAI citation (e.g. a EuCNC/WCNC demo paper) vs. the project URL.

[6] O-RAN Alliance, "O-RAN working group 3, near-RT RIC architecture and E2 general aspects and principles (E2AP)," O-RAN Alliance, Tech. Rep., 2023, vERIFY: exact spec number/version (e.g. O-RAN.WG3.E2AP-v03.00) and year.

[7] ——, "O-RAN working group 3, E2 service model (E2SM) KPM," O-RAN Alliance, Tech. Rep., 2023, vERIFY: exact spec number/version (e.g. O-RAN.WG3.E2SM-KPM-v03.00) and year.

[8] M. Fiedler, T. Hossfeld, and P. Tran-Gia, "The IQX hypothesis: A fundamental property of quality of experience," *IEEE Network*, 2010, vERIFY: exact volume/issue/page numbers.

[9] ITU-T, "P.1203: Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport," ITU-T, Tech. Rep., 2017, vERIFY: exact edition/amendment referenced (base P.1203 vs. P.1203.1/.2/.3).